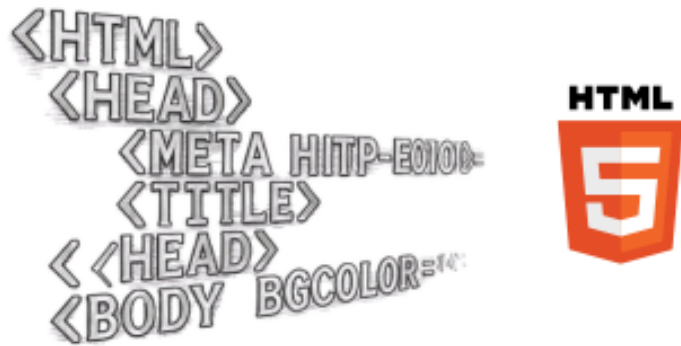




HTML - Complete Notes

What is HTML?

HTML stands for:

HyperText Markup Language

Let's understand each word:

HyperText

HyperText refers to text that contains links to other documents or web pages.

Example:

When you click a link on a website and move to another page, you are using HyperText.

Markup

Markup means using tags to structure content.

Example:

`<h1>Ekta Verma</h1>`

The `<h1>` tag tells the browser that the content is a heading.

Language

HTML is called a markup language because it follows specific syntax and rules.

Definition of HTML

HTML is the standard markup language used to create and structure web pages.

It defines:

1. Headings
2. Paragraphs
3. Images
4. Links
5. Tables
6. Forms
7. Multimedia
8. Page Layout

HTML does NOT:

- Style pages (CSS does)
 - Add functionality (JavaScript does)
-

Why HTML is Important?

Every website on the internet uses HTML.

Examples:

- Google
- Amazon
- Facebook
- YouTube
- Netflix

Without HTML, browsers cannot understand webpage content.

Think of HTML as the skeleton of a website.

Example:

Human Body

- Bones = HTML
- Skin & Appearance = CSS
- Movements = JavaScript

Website

- Structure = HTML
 - Design = CSS
 - Functionality = JavaScript
-

History of HTML

Before HTML existed, sharing information across computers was difficult.

Scientists needed a way to connect documents and information through hyperlinks.

The solution was HTML.

Who Invented HTML?

HTML was invented by:

Tim Berners-Lee

Year:

1991

He is also known as:

“The Father of the World Wide Web”

While working at CERN (European Organization for Nuclear Research), he developed:

- HTML
- HTTP
- URL

These three technologies became the foundation of the modern web.

Evolution of HTML

HTML 1.0 (1991)

First version of HTML.

Features:

- Basic text
- Links
- Headings
- Paragraphs

Limitations:

- Very limited functionality
- No styling support

Example:

```
<h1>Hello World</h1>
```

HTML 2.0 (1995)

First official HTML standard.

New Features:

- Forms
- Input fields
- Buttons

Example:

```
<form>
<input type="text">
</form>
```

Major Improvement:

Allowed user interaction.

HTML 3.2 (1997)

Introduced:

- Tables
- Applets
- Improved formatting

Example:

```
<table>
<tr>
<td>Data</td>
</tr>
</table>
```

Problem:

Many styling tags were mixed into HTML.

HTML 4.01 (1999)

Major milestone.

New Features:

- Better forms
- CSS support
- Accessibility improvements

Example:

```
<link rel="stylesheet" href="style.css">
```

Important Change:

Presentation moved from HTML to CSS.

XHTML (2000)

XHTML stands for:

Extensible HyperText Markup Language

Characteristics:

- Stricter syntax
- XML based

Example:

```
<br />  

```

Every tag needed proper closing.

Although useful, it was difficult for beginners.

HTML5 (2014)

Current standard used today.

Most important version.

Introduced:

- Audio
- Video
- Canvas
- Local Storage
- Semantic Elements
- Geolocation

Example:

```
<video controls>  
</video>  
  
<audio controls>  
</audio>
```

Semantic Elements:

```
<header>  
<nav>  
<main>  
<section>  
<footer>
```

Benefits:

- Faster websites

- Better SEO
 - Better accessibility
 - Mobile-friendly development
-

HTML Versions Summary

Version	Year	Features
HTML 1.0	1991	Basic Web Pages
HTML 2.0	1995	Forms
HTML 3.2	1997	Tables
HTML 4.01	1999	CSS Support
XHTML	2000	Strict Rules
HTML5	2014	Multimedia, Semantic Tags

How HTML Works?

Step 1:

Developer writes HTML code.

Example:

```
<h1>Hello World</h1>
```

Step 2:

Browser reads the code.

Step 3:

Browser converts code into a visual webpage.

Output:

Hello World

Browser Examples:

1. Chrome
 2. Firefox
 3. Edge
 4. Safari
-

HTML Document Structure

Every HTML document follows a standard structure.

Basic Structure:

```
<!DOCTYPE html>
<html>
<head>
<title>My Website</title>
</head>

<body>
<h1>Welcome</h1>
</body>

</html>
```

Let's understand every part.

DOCTYPE Declaration

```
<!DOCTYPE html>
```

Purpose:

Tells browser:

“This document uses HTML5.”

Must always be the first line.

Without it:

Browser may enter Quirks Mode and display pages incorrectly.

HTML Root Element

```
<html>
</html>
```

Represents the root of the webpage.

Everything must be inside this element.

Example:

```
<html>

<head>
</head>

<body>
</body>
```

`</html>`

Head Section

`<head>`
`</head>`

Contains information about the webpage.

Not visible to users.

Common Content:

- Title
- Meta Tags
- CSS Links
- Favicon

Example:

```
<head>

<title>Portfolio</title>

<meta charset="UTF-8">

</head>
```

Title Element

`<title>My Website</title>`

Purpose:

Displays page title on browser tab.

Example:

Browser Tab:

My Website

Importance:

- SEO
 - User Experience
-

Body Section

```
<body>  
</body>
```

Contains visible content.

Examples:

- Headings
- Paragraphs
- Images
- Forms
- Tables

Example:

```
<body>  
  
<h1>Welcome</h1>  
  
<p>Learning HTML</p>  
  
</body>
```

Complete Document Structure Example

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
<title>HTML Training</title>  
  
</head>  
  
<body>  
  
<h1>Welcome to HTML</h1>  
  
<p>This is my first webpage.</p>  
  
</body>  
  
</html>
```

HTML Comments

Comments are notes written inside code.

Browsers ignore comments.

Comments are useful for:

- Documentation
 - Explanations
 - Debugging
 - Team collaboration
-

Syntax of Comments

```
<!-- This is a comment -->
```

Example:

```
<!-- Main Header -->
```

```
<h1>Welcome</h1>
```

Browser Output:

Welcome

Comment is not displayed.

Why Use Comments?

1. Explain Code

```
<!-- Navigation Menu -->  
<nav>  
...  
</nav>
```

2. Organize Sections

```
<!-- Header Section -->  
  
<!-- Main Content -->  
  
<!-- Footer Section -->
```

3. Temporary Disable Code

```
<!--  
  
<h1>Old Heading</h1>  
  
-->
```

Browser ignores it.

Useful for testing.

Multi-Line Comments

```
<!--
```

```
Student Portal
```

```
Created by Anand
```

```
Version 1.0
```

```
-->
```

Best Practices for Comments

Good:

```
<!-- Student Registration Form -->
```

Bad:

```
<!-- form -->
```

Comments should clearly explain purpose.

Interview Questions

Q1. What is HTML?

HTML stands for HyperText Markup Language and is used to structure web pages.

Q2. Who invented HTML?

Tim Berners-Lee in 1991.

Q3. What is the latest version of HTML?

HTML5.

Q4. What is the purpose of DOCTYPE?

It tells the browser that the document follows HTML5 standards.

Q5. Difference between Head and Body?

Head:

Contains metadata and page information.

Body:

Contains visible webpage content.

Q6. Are comments visible in browser?

No.

Comments are ignored by browsers.

Key Takeaways

1. HTML is the foundation of every website.
 2. Tim Berners-Lee invented HTML in 1991.
 3. HTML5 is the current standard.
 4. Every webpage follows a document structure.
 5. DOCTYPE is mandatory.
 6. Head contains metadata.
 7. Body contains visible content.
 8. Comments help developers understand and maintain code.
-

HTML Text Elements & Text Formatting Elements

Introduction

Text is the most important content on a webpage. HTML provides various elements to display, structure, emphasize, highlight, and semantically define text.

Text elements can be divided into:

9. Text Content Elements
 10. Text Formatting Elements
 11. Semantic Text Elements
-

1. Headings

HTML provides six levels of headings.

```
<h1>Main Heading</h1>
<h2>Sub Heading</h2>
<h3>Section Heading</h3>
<h4>Sub Section</h4>
<h5>Minor Heading</h5>
<h6>Smallest Heading</h6>
```

Purpose

- Define document hierarchy
- Improve SEO
- Improve accessibility

Best Practice

Only one H1 per page whenever possible.

Example:

```
<h1>MERN Stack Training</h1>
<h2>HTML Module</h2>
<h3>Text Formatting</h3>
```

2. Paragraph Element

```
<p>This is a paragraph.</p>
```

Purpose:

- Represents a block of text
- Automatically creates spacing

Example:

```
<p>HTML is the standard markup language for creating web pages.</p>
```

3. Line Break

```
<br>
```

Example:

Name: Anand

Course: MERN Stack

Output:

Name: Anand Course: MERN Stack

4. Horizontal Rule

```
<hr>
```

Purpose:

Creates a thematic break.

Example:

```
<p>Section 1</p>  
<hr>  
<p>Section 2</p>
```

TEXT FORMATTING ELEMENTS

These elements modify or provide meaning to text.

5. Bold Text

b Element

```
<b>Bold Text</b>
```

Purpose:

Visual bold appearance only.

Example:

```
<p>This is <b>bold</b> text.</p>
```

strong Element

```
<strong>Important Text</strong>
```

Purpose:

Indicates strong importance.

Example:

```
<p>Please <strong>save your work</strong>.</p>
```

Difference:

b = Visual styling

strong = Important meaning

6. Italic Text

i Element

```
<i>Italic Text</i>
```

Used for:

- Foreign words
- Scientific names
- Book titles
- Technical terms

Example:

```
<p>I love <i>Harry Potter</i>.</p>
```

em Element

```
<em>Emphasized Text</em>
```

Purpose:

Indicates stress or emphasis.

Example:

```
<p>You <em>must</em> complete this task.</p>
```

Difference:

i = Different text

em = Important stressed text

7. Underline

```
<u>Underlined Text</u>
```

Example:

```
<p><u>Important Notice</u></p>
```

Use carefully because users often confuse it with links.

8. Highlight Text

```
<mark>Highlighted Text</mark>
```

Example:

```
<p>HTML is <mark>important</mark>.</p>
```

Used in:

- Search results
 - Notes
 - Tutorials
-

9. Small Text

`<small>Small Text</small>`

Example:

`<small>Terms and Conditions Apply</small>`

Common Uses:

- Copyright
 - Disclaimers
 - Footnotes
-

10. Deleted Text

`<del>Deleted Text</del>`

Example:

`<p>Price: <del>₹1000</del> ₹800</p>`

Used in:

- E-commerce websites
 - Revision history
(horizontal line over text)
-

11. Inserted Text

`<ins>Inserted Text</ins>`

Example:

`<p>New Price: <ins>₹800</ins></p>`

Represents newly inserted content.(underlined)

12. Strikethrough

`<s>Not Relevant</s>`

Example:

`<p><s>Offer Expired</s></p>`

(horizontal line over text)

Difference:

s = No longer relevant

del = Deleted content

13. Subscript

H₂O

Output:

H₂O

Used in:

- Chemistry
 - Mathematics
-

14. Superscript

x²

Output:

x²

Used in:

- Exponents
 - Footnotes
-

15. Code Element

<code>console.log()</code>

Purpose:

Represents programming code.

Example:

<p>Use <code>npm install</code></p>

16. Keyboard Input

<kbd>Ctrl + C</kbd>

Used for keyboard shortcuts.

Example:

Press <kbd>Ctrl</kbd> + <kbd>S</kbd>

17. Sample Output

`<samp>Login Successful</samp>`

Represents output generated by a program.

18. Variable Element

`<var>x</var>`

Represents a variable.

Example:

`<var>x</var> + <var>y</var>`

19. Preformatted Text

`<pre>`

Name: Ekta

Course: MERN

`</pre>`

Preserves:

- Spaces
 - Tabs
 - Line breaks
-

20. Inline Quote

`<q>Knowledge is power.</q>`

Short quotation.(in double quotes)

21. Block Quote

`<blockquote>`

Knowledge is power.

`</blockquote>`

Long quotation.(not in double quotes)

SEMANTIC TEXT ELEMENTS

These provide meaning to content.

22. Abbreviation

```
<abbr title="HyperText Markup Language">  
HTML  
</abbr>
```

Purpose:

Defines abbreviations.(dotted underline)

Examples:

HTML CSS JS WHO

Benefits:

- Accessibility
 - Better understanding
-

23. Citation

```
<cite>Rich Dad Poor Dad</cite>
```

(slanted text)

Used for:

- Books
 - Movies
 - Research Papers
 - Articles
-

24. Address

```
<address>  
John Gomez  
JohnGomez@gmail.com  
</address>
```

Purpose:

Represents contact information.

Usually placed in:

- Footer
 - Contact Page
 - Author Information
-

25. Time Element

```
<time datetime="2026-06-05">  
8 May 2000  
</time>
```

Benefits:

- SEO
 - Machine-readable dates
-

26. Bidirectional Override

```
<bdo dir="rtl">  
HELLO  
</bdo>
```

Purpose:

Changes text direction.

Rarely used.

Interview Questions

Q1. Difference between b and strong?

Answer: b provides visual bold styling. strong provides importance and semantic meaning.

Q2. Difference between i and em?

Answer: i is used for alternate voice or different text. em provides emphasis and importance.

Q3. Difference between del and s?

Answer: del indicates deleted content. s indicates content that is no longer relevant.

Q4. Difference between q and blockquote?

Answer: q is for short inline quotations. blockquote is for long quotations.

Q5. Difference between cite and address?

Answer: cite represents the title of a creative work. address represents contact information.

Best Practices

5. Prefer semantic tags over purely visual tags.
6. Use strong instead of b when text is important.

7. Use `em` instead of `i` when emphasizing text.
 8. Use proper heading hierarchy.
 9. Use `abbr` for abbreviations.
 10. Use `time` for dates and times.
 11. Avoid excessive formatting.
-

Summary

Most Important Tags for Real Projects

Headings Paragraph strong em mark small del sub sup code pre abbr cite address time

These tags improve SEO, accessibility, readability, and semantic correctness of modern HTML documents.

HTML Attributes, Hyperlinks (Links) and Images Elements

HTML Attributes

What is an Attribute?

Attributes provide additional information about an HTML element.

Attributes are always written inside the opening tag.

Syntax:

```
<tagname attribute="value">
```

Example:

```
<a href="https://google.com">  
Google  
</a>
```

Here:

12. `href` = Attribute

13. `https://google.com` = Value

Why Do We Need Attributes?

Without attributes:

```
<a>Google</a>
```

Browser knows it's a link but doesn't know where to go.

With attributes:

```
<a href="https://google.com">  
Google  
</a>
```

Now browser knows the destination.

Parts of an Attribute

Example:

```

```

Part	Meaning
src	Attribute Name
logo.png	Attribute Value
alt	Attribute Name
Company Logo	Attribute Value

Types of Attributes

Global Attributes

Can be used on almost every HTML element.

Examples:

- id
 - class
 - style
 - title
 - lang
 - hidden
 - draggable
-

Element Specific Attributes

Used only on specific elements.

Example:

```
<a href="">
```

href works only for links.

```
<img src="">
```

src works for images.

Common Global Attributes

id

Provides a unique identifier.

```
<p id="intro">  
Welcome  
</p>
```

Rules:

- Must be unique
- No spaces

Used for:

- CSS
 - JavaScript
 - Navigation
-

class

Groups elements.

```
<p class="text">  
Paragraph  
</p>
```

Multiple elements can share same class.

title

Displays tooltip.

```
<p title="This is tooltip">  
Hover me  
</p>
```

style

Inline CSS.

```
<p style="color:red">  
Hello  
</p>
```

lang

Defines page language.

```
<html lang="en">
```

Benefits:

- SEO
- Accessibility

hidden

Hides element.

```
<p hidden>  
Hidden Content  
</p>
```

draggable

Makes element draggable.

```
<p draggable="true">  
Drag Me  
</p>
```

HTML Links (Hyperlinks)

What is a Hyperlink?

A hyperlink connects one resource to another.

Examples:

- Website → Website
- Page → Page
- Page → Image
- Page → PDF
- Page → Email

Hyperlinks are created using:

```
<a>
```

Anchor Tag

Basic Link Syntax

```
<a href="https://google.com">  
Visit Google  
</a>
```

Output:

Visit Google

href Attribute

Most important link attribute.

href means:

Hypertext Reference

Example:

```
<a href="https://www.google.com">  
Google  
</a>
```

Defines destination URL.

Types of Links

External Link

```
<a href="https://www.amazon.in">  
Amazon  
</a>
```

Navigates to another website.

Internal Link

```
<a href="about.html">  
About Us  
</a>
```

Navigates within same website.

Relative URL

```
<a href="contact.html">  
Contact  
</a>
```

Uses current website location.

Absolute URL

```
<a href="https://www.google.com">  
Google  
</a>
```

Contains full URL.

target Attribute

Controls where link opens.

_self (Default)

```
<a href="about.html" target="_self">  
About  
</a>
```

Opens in same tab.

_blank

```
<a href="https://google.com" target="_blank">  
Google  
</a>
```

Opens in new tab.

Most commonly used.

_parent

Used inside frames.

_top

Loads page in full browser window.

rel Attribute

Important for SEO and security.

Example:

```
<a href="https://google.com"
target="_blank"
rel="noopener noreferrer">
Google
</a>
```

Benefits:

- Better security
 - Better performance
-

title Attribute

```
<a href="about.html"
title="About Our Company">
About
</a>
```

Tooltip appears on hover.

Download Link

```
<a href="resume.pdf" download>
Download Resume
</a>
```

Forces download.

Email Link

```
<a href="mailto:abc@gmail.com">
Email Us
</a>
```

Opens email application.

Phone Link

```
<a href="tel:+919999999999">
Call Now
</a>
```

Useful on mobile devices.

WhatsApp Link

```
<a href="https://wa.me/919999999999">  
Chat on WhatsApp  
</a>
```

Image Link

```
<a href="https://google.com">  
  
  
  
</a>
```

Image becomes clickable.

Page Navigation (Bookmark Links)

Navigate to sections within same page.

Section:

```
<h2 id="contact">  
Contact  
</h2>
```

Link:

```
<a href="#contact">  
Go To Contact  
</a>
```

Very common in landing pages.

HTML Images

Why Images Are Important?

Images improve:

- User Experience
- Engagement
- Branding
- SEO

Images are displayed using:

```
<img>
```

Image Syntax

```

```

Image tag is self-closing.

src Attribute

Most important image attribute.

src = Source

```

```

Tells browser where image is located.

Image Paths

Same Folder

```

```

Images Folder

```

```

External Image

```

```

alt Attribute

Most important image attribute.

Example:

```

```

Purpose:

- Accessibility
- SEO
- Fallback text

If image fails:

Company Logo appears.

width Attribute

```

```

Sets width.

height Attribute

```

```

Sets height.

title Attribute

```

```

Tooltip on hover.

loading Attribute

HTML5 feature.

```

```

Values:

- lazy
- eager

Benefits:

- Faster websites
 - Better performance
-

Image as Link

```
<a href="index.html">  
  
  
  
</a>
```

figure Element

Used to group image and caption.

```
<figure>  
  
  
  
<figcaption>  
Beautiful Nature  
</figcaption>  
  
</figure>
```

Semantic HTML.

Responsive Images

```

```

Or use CSS:

```
img{  
max-width:100%;  
height:auto;  
}
```

Image Formats

JPEG (.jpg)

Best for:

- Photos

Advantages:

- Small size

PNG (.png)

Best for:

- Logos
- Transparent images

SVG (.svg)

Best for:

12. Icons
13. Vector graphics

Advantages:

- Infinite scaling
- Very small size

WebP (.webp)

Modern image format.

Advantages:

- Smaller size
- Better performance

Recommended today.

SEO Best Practices for Images

Bad:

```

```

Good:

```

```

Benefits:

- Better SEO
 - Better Accessibility
-

Interview Questions

What is an Attribute?

Attributes provide additional information about HTML elements.

Difference Between Absolute and Relative URL?

Absolute:

`https://google.com`

Relative:

`about.html`

What is href?

Defines the destination of a hyperlink.

What is target="_blank"?

Opens link in new tab.

Why is alt attribute important?

- Accessibility
 - SEO
 - Fallback text
-

Difference Between title and alt?

title:

Tooltip on hover.

alt:

Alternative text when image fails.

Which image format is best for logos?

PNG or SVG

Which image format is best for photographs?

JPEG or WebP

Key Takeaways

1. Attributes provide additional information to elements.
 2. href is the most important link attribute.
 3. src is the most important image attribute.
 4. alt is mandatory for accessibility and SEO.
 5. Use target="_blank" with rel="noopener noreferrer".
 6. Prefer WebP and SVG for modern websites.
 7. Use meaningful file names and alt text for better SEO.
-

HTML Lists and Tables Elements

HTML Lists

What is a List?

A list is a collection of related items displayed in a structured format.

Examples:

Shopping List

- 14. Milk
- 15. Bread
- 16. Butter

Course Modules

- HTML
- CSS
- JavaScript

Lists improve:

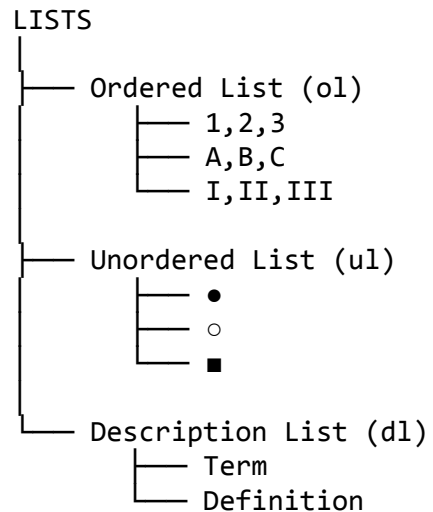
- Readability
 - Organization
 - SEO
 - User Experience
-

Types of Lists in HTML

HTML provides three types of lists:

- Ordered List (ol)
 - Unordered List (ul)
 - Description List (dl)
-

Visual Overview



Unordered List

Used when order does not matter.

Syntax:

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

Output:

- HTML
 - CSS
 - JavaScript
-

ul Element

Container of unordered list.

```
<ul>
</ul>
```

li Element

Represents a list item.

```
<li>Item</li>
```

Every list must contain li elements.

Nested Unordered List

```
<ul>
```

```
<li>Frontend
```

```
  <ul>
    <li>HTML</li>
    <li>CSS</li>
  </ul>
```

```
</li>
```

```
</ul>
```

Output:

```
Frontend
├── HTML
└── CSS
```

Ordered List

Used when sequence matters.

Examples:

- Instructions
- Recipes
- Exam Rankings

Syntax:

```
<ol>
```

```
<li>Install Node.js</li>
<li>Install VS Code</li>
<li>Create Project</li>
```

`</ol>`

Output:

- Install Node.js
 - Install VS Code
 - Create Project
-

Ordered List Attributes

type Attribute

Changes numbering style.

Numbers

`<ol type="1">`

Output:

1,2,3

Uppercase Letters

`<ol type="A">`

Output:

A,B,C

Lowercase Letters

`<ol type="a">`

Output:

a,b,c

Uppercase Roman

`<ol type="I">`

Output:

I,II,III

Lowercase Roman

```
<ol type="i">
```

Output:

i,ii,iii

start Attribute

Changes starting value.

```
<ol start="5">
```

```
<li>HTML</li>
```

```
<li>CSS</li>
```

```
</ol>
```

Output:

- HTML
 - CSS
-

reversed Attribute

Displays reverse numbering.

```
<ol reversed>
```

Output:

3

2

1

Description List

Used for terms and definitions.

Syntax:

```
<dl>
```

```
<dt>HTML</dt>
```

```
<dd>HyperText Markup Language</dd>
```

```
</dl>
```

Output:

HTML

HyperText Markup Language

Elements Used

dl

Description List Container

```
<dl>  
</dl>
```

dt

Description Term

```
<dt>HTML</dt>
```

dd

Description Details

```
<dd>HyperText Markup Language</dd>
```

Real World Example

```
<dl>  
  
<dt>CPU</dt>  
  
<dd>Central Processing Unit</dd>  
  
<dt>RAM</dt>  
  
<dd>Random Access Memory</dd>  
  
</dl>
```

HTML Tables

What is a Table?

Tables display data in rows and columns.

Examples:

Student Records

Employee Data

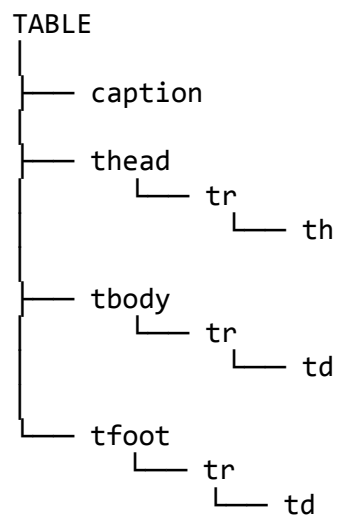
Marksheets

Invoices

Visual Structure

Name	Course	Fee
Aisha	MERN	25000
Rahul	Java	20000

Table Structure Overview



Basic Table Syntax

```
<table>
```

```
<tr>
```

```
<th>Name</th>
```

```
<th>Course</th>
```

```
</tr>
```

```
<tr>
```

```
<td>Aisha</td>
```

```
<td>MERN</td>
```

```
</tr>
```



```
</table>
```

table Element

Main container.

```
<table>  
</table>
```

tr Element

Table Row

```
<tr>  
</tr>
```

Every row uses tr.

th Element

Table Heading Cell

```
<th>Name</th>
```

Output:

Bold and centered.

td Element

Table Data Cell

```
<td>Aisha</td>
```

Stores actual data.

Complete Example

```
<table>  
  
<tr>  
<th>Name</th>  
<th>Course</th>  
</tr>
```

```
<tr>
<td>Aisha</td>
<td>MERN</td>
</tr>

<tr>
<td>Rahul</td>
<td>Java</td>
</tr>

</table>
```

Caption Element

Adds table title.

```
<table>

<caption>
Student Information
</caption>

</table>
```

Output:

Student Information

(Table below)

thead Element

Header section.

```
<thead>

<tr>
<th>Name</th>

</tr>

</thead>
```

Used for:

- Accessibility
 - Better structure
-

tbody Element

Main data section.

```
<tbody>

<tr>
<td>Anand</td>
</tr>

</tbody>
```

tfoot Element

Footer section.

```
<tfoot>

<tr>
<td>Total</td>
</tr>

</tfoot>
```

Useful in reports.

colspan Attribute

Merge columns.

Example:

```
<td colspan="2">
Merged Cell
</td>
```

Visual:

```
+-----+
| Merged Cell |
+-----+
```

rowspan Attribute

Merge rows.

Example:

```
<td rowspan="2">
MERN
</td>
```

Visual:

MERN

Example Using colspan

```
<table>

<tr>
<td colspan="2">
Student Information
</td>
</tr>

</table>
```

Example Using rowspan

```
<table>

<tr>
<td rowspan="2">
MERN
</td>

<td>HTML</td>
</tr>

<tr>
<td>CSS</td>
</tr>

</table>
```

Common Table Attributes

border

```
<table border="1">
```

Creates border.

Note:

Modern websites use CSS instead.

cellpadding

Space inside cells.

```
<table cellpadding="10">
```

cellspacing

Space between cells.

```
<table cellspacing="5">
```

width

```
<table width="500">
```

height

```
<table height="300">
```

Modern Recommendation

Instead of:

```
<table border="1">
```

Use CSS:

```
table{  
border:1px solid black;  
}
```

Real World Student Table

```
<table>
```

```
<caption>
```

Student Details

```
</caption>
```

```
<thead>
```

```
<tr>
```

```
<th>Name</th>
```

```
<th>Course</th>
```

```
<th>Fee</th>
</tr>

</thead>

<tbody>

<tr>
<td>Aisha</td>
<td>MERN</td>
<td>25000</td>
</tr>

<tr>
<td>Rahul</td>
<td>Java</td>
<td>20000</td>
</tr>

</tbody>

<tfoot>

<tr>
<td colspan="2">
Total Students
</td>

<td>2</td>
</tr>

</tfoot>

</table>
```

Interview Questions

Difference Between ul and ol?

ul:

Order does not matter.

ol:

Order matters.

Difference Between th and td?

th:

Heading cell

td:

Data cell

Difference Between thead and tbody?

thead:

Table header

tbody:

Table data

What is colspan?

Merges multiple columns.

What is rowspan?

Merges multiple rows.

What is caption?

Provides table title.

Best Practices

- Use ul for menus and navigation.
 - Use ol for step-by-step instructions.
 - Use dl for definitions and glossaries.
 - Always use thead, tbody and tfoot for large tables.
 - Use caption for accessibility.
 - Use CSS instead of deprecated table attributes.
 - Never use tables for page layout.
-

Key Takeaways

Lists:

- ul
- ol
- li
- dl

- dt
- dd

Tables:

- table
- tr
- th
- td
- caption
- thead
- tbody
- tfoot
- colspan
- rowspan

These elements are heavily used in forms, dashboards, admin panels, reports, e-commerce websites and enterprise applications.

HTML Forms and Multimedia Elements

HTML Forms

What is a Form?

A form is used to collect data from users.

Examples:

17. Login Form
18. Registration Form
19. Contact Form
20. Feedback Form
21. Job Application Form
22. Online Exam Form

Without forms, websites cannot collect information from users.

Real World Examples

Instagram Login

Username: _____

Password: _____

[Login]

Student Registration Form

Name

Email

Phone

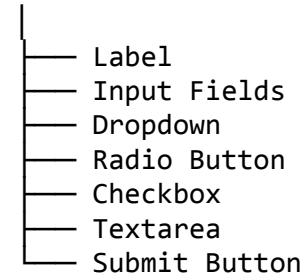
Course

Submit

All these are forms.

Form Structure

FORM



form Element

The form tag is the container of all form controls.

Syntax:

```
<form>
```

```
</form>
```

Example:

```
<form>
```

```
<input type="text">
```

```
</form>
```

Important Form Attributes

action

Specifies where form data will be sent.

```
<form action="save.php">
```

Example:

```
<form action="/register">
```

After submission:

Data goes to /register

method

Defines how data is sent.

Values:

- GET
- POST

Example:

```
<form method="POST">
```

GET Method

```
<form method="GET">
```

Data appears in URL.

Example:

<https://website.com?name=Anand>

Advantages:

- Easy testing

Disadvantages:

- Less secure

Use For:

- Search Forms
-

POST Method

```
<form method="POST">
```

Data hidden from URL.

Advantages:

- More secure

Use For:

- Login
 - Registration
 - Payment
-

Label Element

Associates text with input.

```
<label>Name</label>
```

Better Example:

```
<label for="name">  
Name  
</label>
```

```
<input id="name">
```

Benefits:

- Accessibility
 - Better UX
-

Input Element

Most important form element.

Syntax:

```
<input>
```

Input types determine behavior.

Input Attributes

name

Used to identify field.

```
<input  
name="username">
```

Data sent to server:

username=Akansha

value

Default value.

```
<input  
value="Akansha">
```

placeholder

Hint text.

```
<input  
placeholder="Enter Name">
```

Output:

Enter Name...

required

Mandatory field.

```
<input required>
```

Cannot submit empty form.

readonly

Cannot edit value.

```
<input  
readonly>
```

disabled

Disables field completely.

```
<input  
disabled>
```

maxlength

Maximum characters.

```
<input  
maxlength="10">
```

minlength

Minimum characters.

```
<input  
minlength="5">
```

Input Types

type="text"

```
<input type="text">
```

Used for:

- Name
 - City
 - Course
-

type="password"

```
<input type="password">
```

Masks characters.

Output:

type="email"

```
<input type="email">
```

Validates email format.

Example:

abc@gmail.com

type="number"

```
<input type="number">
```

Allows numeric input.

Attributes:

min
max
step

Example:

```
<input  
type="number"  
min="1"  
max="100">
```

type="tel"

```
<input type="tel">
```

```
<form action="">  
  
  <input type="tel"  
    name="mobile"  
    pattern="[0-9]{10}"  
    placeholder="Enter 10-digit mobile number"  
    required>  
  
  <input type="submit">  
  
</form>
```

Phone numbers.

type="url"

```
<input type="url">
```

```
<form action="">
```

```
<input type="url"

      id="github"

      name="github"

      placeholder="https://github.com/username"

      required>

<input type="submit">

</form>
```

Website URL validation.

type="date"

```
<input type="date">
```

Date picker.

type="time"

```
<input type="time">
```

Time picker.

type="datetime-local"

```
<input
type="datetime-local">
```

Date + Time.

type="month"

```
<input type="month">
```

type="week"

```
<input type="week">
```

type="color"

```
<input type="color">
```

Color picker.

type="range"

```
<input  
type="range"  
min="0"  
max="100">
```

Slider.

type="file"

```
<input type="file">
```

Used for uploads.

Common Attributes:

accept
multiple

Example:

```
<input  
type="file"  
accept=".pdf">
```

type="radio"

Used when only one option can be selected.

```
<input  
type="radio"  
name="gender">
```

Example:

Male
Female

Only one selectable.

type="checkbox"

Used when multiple selections allowed.

```
<input  
type="checkbox">
```

Example:

HTML
CSS
JavaScript

Multiple selectable.

type="submit"

```
<input type="submit">
```

Submits form.

type="reset"

```
<input type="reset">
```

Clears form.

type="button"

```
<input type="button">
```

General purpose button.

Textarea Element

Multi-line text input.

```
<textarea>  
  
</textarea>
```

Example:

```
<textarea  
rows="5"  
cols="30">  
</textarea>
```

Used for:

- Comments
 - Feedback
 - Address
-

Select Element

Dropdown list.

```
<select>  
<option>HTML</option>  
<option>CSS</option>  
</select>
```

Select Attributes

multiple

```
<select multiple>
```

Allows multiple selections.

size

```
<select size="4">
```

Visible options count.

Option Element

```
<option>  
HTML  
</option>
```

Attributes:

selected

```
<option selected>  
HTML  
</option>
```

Default selected.

Fieldset Element

Groups related fields.

```
<fieldset>
```

```
</fieldset>
```

Legend Element

Title of fieldset.

```
<legend>
```

Personal Information

```
</legend>
```

Complete Registration Form Structure

FORM

```
|
|—— Name
|—— Email
|—— Password
|—— Gender
|—— Skills
|—— Address
|—— Course
|—— Submit
```

HTML5 Validation Attributes

required

pattern

maxlength

minlength

min

max

step

Example:

```
<input
type="text"
required
maxlength="20">
```

Multimedia Elements

What is Multimedia?

Multimedia means:

Multiple forms of media.

Examples:

- Images
- Audio
- Video
- Animations

HTML5 introduced native multimedia support.

Before HTML5:

Plugins like Flash were required.

Now browsers support multimedia directly.

Multimedia Elements Overview

MULTIMEDIA

- img
- audio
- video
- source
- track
- figure
- iframe

Audio Element

Used to play audio.

Syntax:

```
<audio controls>
```

```
</audio>
```

Basic Audio Example

```
<audio controls>
```

```
<source  
src="song.mp3"  
type="audio/mpeg">  
  
</audio>
```

Audio Attributes

controls

```
<audio controls>
```

Displays player controls.

autoplay

```
<audio autoplay>
```

Starts automatically.

loop

```
<audio loop>
```

Repeats audio.

muted

```
<audio muted>
```

Starts muted.

Video Element

Used to play videos.

Syntax:

```
<video controls>
```

```
</video>
```

Basic Video Example

```
<video  
width="500"  
controls>
```

```
<source  
src="video.mp4"  
type="video/mp4">  
  
</video>
```

Video Attributes

controls

Shows player controls.

autoplay

Starts automatically.

muted

Required for autoplay in most browsers.

loop

Repeats video.

poster

Thumbnail image.

```
<video  
poster="thumbnail.jpg">
```

width

width="500"

height

height="300"

Source Element

Allows multiple formats.

```
<video controls>
```

```
<source  
src="movie.mp4"  
type="video/mp4">
```

```
<source  
src="movie.webm"  
type="video/webm">
```

```
</video>
```

Browser selects supported format.

Track Element

Adds subtitles.

```
<track  
src="subtitle.vtt"  
kind="subtitles">
```

Used for:

- Captions
 - Accessibility
-

Iframe Element

Embeds external content.

Examples:

- YouTube Videos
 - Google Maps
 - External Websites
-

YouTube Video Embed

```
<iframe  
width="560"  
height="315"  
src="https://www.youtube.com/embed/xyz">  
</iframe>
```

Google Maps Embed

```
<iframe  
src="google-map-url">  
</iframe>
```

Figure and Figcaption

```
<figure>  
  
  
  
<figcaption>  
Beautiful Nature  
</figcaption>  
  
</figure>
```

Provides semantic grouping.

Common Multimedia Formats

Audio

- 14. MP3
 - 15. WAV
 - 16. OGG
-

Video

- MP4
- WebM
- OGG

Recommended:

MP4

Most supported.

Interview Questions

Difference Between GET and POST?

GET:

Data visible in URL.

POST:

Data hidden from URL.

Difference Between Radio and Checkbox?

Radio:

Single selection.

Checkbox:

Multiple selection.

Difference Between Button and Submit?

Button:

No action by default.

Submit:

Submits form.

Why use Label?

Improves accessibility and user experience.

Difference Between Audio and Video?

Audio:

Sound only.

Video:

Sound + Visuals.

Why use Source Tag?

Supports multiple media formats.

What is Poster Attribute?

Thumbnail image displayed before video starts.

Why is enctype required?

Required when uploading files using forms.

Best Practices

1. Always use labels with inputs.
 2. Use POST for sensitive information.
 3. Use required validation whenever needed.
 4. Use fieldset and legend for grouping.
 5. Always provide multiple video formats.
 6. Use captions for accessibility.
 7. Avoid autoplay audio.
 8. Use optimized media files.
-

Key Takeaways

Forms:

- form
- input
- textarea
- select
- option
- label
- fieldset
- legend

Multimedia:

- audio
- video
- source
- track
- iframe
- figure
- figcaption

These elements are heavily used in registration systems, dashboards, e-commerce websites, LMS platforms, social media applications, and enterprise web applications.

Semantic HTML and HTML Entities

Semantic HTML

What is Semantic HTML?

Semantic means:

“Providing Meaning”

Semantic HTML uses tags that clearly describe their purpose and content.

Instead of using generic tags like:

```
<div>
```

We use meaningful tags like:

```
<header>
```

```
<nav>
```

```
<main>
```

```
<footer>
```

When another developer sees the code, they immediately understand the purpose of each section.

Why Semantic HTML?

Imagine a webpage built like this:

```
<div class="header">
```

```
</div>
```

```
<div class="menu">
```

```
</div>
```

```
<div class="content">
```

```
</div>
```

```
<div class="footer">
```

```
</div>
```

The browser sees only div tags.

It doesn't know:

23. Which part is navigation

24. Which part is content

25. Which part is footer

Now using Semantic HTML:

```
<header>
```

```
</header>
```

```
<nav>
```

```
</nav>
```

```
<main>
```

```
</main>
```

```
<footer>
```

```
</footer>
```

Everything becomes meaningful.

Benefits of Semantic HTML

Better Readability

Code becomes easier to understand.

Bad:

```
<div class="top">
```

Good:

```
<header>
```

Better SEO

Search engines understand content better.

Google can identify:

- Main Content
- Navigation
- Articles
- Footer

Result:

Better ranking opportunities.

Better Accessibility

Screen readers can identify page sections.

Helpful for:

- Blind users
 - Visually impaired users
-

Easier Maintenance

Developers understand structure quickly.

Semantic vs Non-Semantic Elements

Non-Semantic Elements

These do not describe content meaning.

Examples:

```
<div>  
<span>
```

Browser doesn't know their purpose.

Semantic Elements

Examples:

```
<header>  
<nav>  
<main>  
<section>  
<article>  
<aside>  
<footer>
```

Purpose is clear.

Semantic Layout Structure

Visual Representation

```
+-----+
|      HEADER      |
+-----+
```

```
+-----+
|      NAV         |
+-----+
```

```
+-----+-----+
| SECTION | ASIDE |
| ARTICLE | Sidebar |
+-----+-----+
```

```
+-----+
|      FOOTER      |
+-----+
```

Important Semantic Elements

Header Element

Represents introductory content.

Syntax:

```
<header>
```

```
</header>
```

Contains:

- Logo
- Website Title
- Search Bar
- Navigation

Example:

```
<header>
```

```
<h1>MERN Stack Training</h1>
```

```
</header>
```

Navigation Element

`<nav>`

`</nav>`

Represents navigation links.

Example:

`<nav>`

`<a href="">Home</a>`

`<a href="">About</a>`

`<a href="">Contact</a>`

`</nav>`

Main Element

Represents primary content.

`<main>`

`</main>`

Important Rule:

One main element per page.

Example:

`<main>`

`<h2>Latest Courses</h2>`

`</main>`

Section Element

Represents a thematic group of content.

Example:

`<section>`

`<h2>About Us</h2>`

`<p>Content</p>`

`</section>`

Think:

A chapter in a book.

Article Element

Represents independent content.

Examples:

- Blog Post
- News Article
- Product Review
- Forum Post

Example:

```
<article>
```

```
<h2>HTML Tutorial</h2>
```

```
<p>Article Content</p>
```

```
</article>
```

Can be distributed independently.

Difference Between Section and Article

Section:

Part of a page.

Article:

Standalone content.

Example:

Website

text id="yq5hlw" SECTION |— ARTICLE 1 |— ARTICLE 2 |— ARTICLE 3

Aside Element

Represents related but secondary content.

Example:

- Sidebar

- Advertisements
- Related Posts
- Recent Articles

<aside>

Related Courses

</aside>

Footer Element

Represents bottom section.

Contains:

- Copyright
- Contact Information
- Social Links
- Privacy Policy

Example:

<footer>

© 2026 Ekta's Training

</footer>

Figure Element

Used for self-contained content.

Usually:

- Images
- Charts
- Diagrams

Example:

<figure>

</figure>

Figcaption Element

Caption for figure.

Example:

```
<figure>



<figcaption>
Beautiful Nature
</figcaption>

</figure>
```

Time Element

Represents date or time.

Example:

```
<time datetime="2026-06-05">

5 June 2026

</time>
```

Benefits:

- SEO
 - Machine Readable
-

Complete Semantic Layout Example

```
<header>

<h1>MERN Academy</h1>

</header>

<nav>

<a href="">Home</a>

<a href="">Courses</a>

</nav>

<main>

<section>
```

`<h2>Frontend</h2>`

`<article>`

`<h3>HTML Course</h3>`

`</article>`

`</section>`

`<aside>`

Upcoming Batches

`</aside>`

`</main>`

`<footer>`

Copyright 2026

`</footer>`

HTML Entities

What are HTML Entities?

Some characters are reserved by HTML.

Example:

`<h1>`

Browser thinks it is a tag.

If we want to display:

text id="uxqhoh" `<h1>`

We must use an entity.

Definition

HTML Entities are special codes used to display reserved characters and symbols.

Syntax:

html id="w7eh8h" &entity_name;

or

```
html id="ot5r1l" &#entity_number;
```

Why Use HTML Entities?

- Display Reserved Characters
 - Display Special Symbols
 - Prevent Parsing Errors
 - Improve Compatibility
-

Reserved Character Entities

Less Than Symbol

Character:

```
text id="f4t04v" <
```

Entity:

```
html id="5m5g0x" &lt;
```

Output:

<

Greater Than Symbol

Character:

```
text id="2hph58" >
```

Entity:

```
html id="92j0j2" &gt;
```

Output:

Ampersand

Character:

```
text id="8iuh4k" &
```

Entity:

```
html id="x5iz6g" &amp;
```

Output:

&

Double Quote

Character:

text id="u2c6qg" "

Entity:

html id="p4zdhk" "

Output:

"

Single Quote

Character:

text id="c8vhdj" '

Entity:

html id="l3np21" '

Output:

,

Non-Breaking Space

Most Important Entity.

Entity:

html id="n6jz0r"

Creates extra space.

Example:

Ekta Verma

Output:

Ekta Verma

Currency Entities

Indian Rupee

`html id="m68n5r" ₹`

Output:

₹

Dollar

`html id="i4nt95" $`

Output:

\$

Euro

`html id="6b64p6" €`

Output:

€

Pound

`html id="hjgf44" £`

Output:

£

Copyright Symbols

Copyright

`html id="njftd5" ©`

Output:

©

Registered Trademark

`html id="k39j8q" ®`

Output:

®

Trademark

```
html id="kv5j2g" &trade;
```

Output:

™

Mathematical Entities

Multiplication

```
html id="svsk92" &times;
```

Output:

×

Division

```
html id="jlwmhf" &divide;
```

Output:

÷

Plus Minus

```
html id="jlwmhg" &plusmn;
```

Output:

±

Degree Symbol

```
html id="jlwmhh" &deg;
```

Output:

°

Example:

```
35&deg;C
```

Output:

35°C

Arrow Entities

`html id="jlwmhi" →`

Output:

→

`html id="jlwmhj" ←`

Output:

←

`html id="jlwmhk" ↑`

Output:

↑

`html id="jlwmhl" ↓`

Output:

↓

Card Suit Symbols

`html id="jlwmhm" ♥`

♥

`html id="jlwmhn" ♠`

♠

Commonly Used Entities Table

Entity	Output
<	<
>	>
&	&
	Space
©	©
®	®
™	™
°	°
×	×

Entity

÷

→

Output

÷

→

Interview Questions

What is Semantic HTML?

Semantic HTML uses meaningful tags that describe content purpose.

Difference Between Div and Section?

div:

Generic container.

section:

Semantic content section.

Difference Between Section and Article?

Section:

Part of webpage.

Article:

Independent content.

Why Use Semantic HTML?

- Better SEO
 - Better Accessibility
 - Better Readability
-

What are HTML Entities?

Special codes used to display reserved characters and symbols.

Why Use < Instead of < ?

Because browser treats < as HTML tag syntax.

What is ?

Non-breaking space.

Used to create additional spacing.

Best Practices

- Use semantic tags whenever possible.
 - Avoid excessive div usage.
 - Use article for standalone content.
 - Use section for grouped content.
 - Use entities for reserved characters.
 - Prefer semantic structure over visual structure.
-

Key Takeaways

Semantic HTML:

- header
- nav
- main
- section
- article
- aside
- footer
- figure
- figcaption
- time

HTML Entities:

17. Reserved Characters

18. Currency Symbols

19. Mathematical Symbols

20. Copyright Symbols

21. Special Spaces

Semantic HTML improves SEO, accessibility, and maintainability, while HTML Entities allow special characters and symbols to be displayed safely inside webpages.

Meta Tags, Favicon and HTML5 Features

Meta Tags

What are Meta Tags?

Meta tags provide information about a webpage.

They are placed inside the:

```
<head>  
</head>
```

section of an HTML document.

Meta tags are generally not visible to users.

Instead, they provide information to:

- 26. Browsers
 - 27. Search Engines
 - 28. Social Media Platforms
 - 29. Web Crawlers
-

Why Meta Tags are Important?

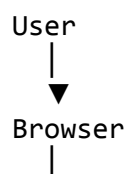
Meta tags help in:

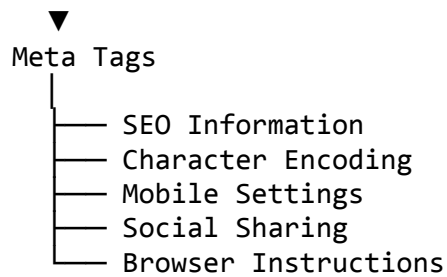
- SEO (Search Engine Optimization)
- Mobile Responsiveness
- Character Encoding
- Browser Compatibility
- Social Sharing

Without meta tags:

- Search engines may not understand the page properly.
 - Mobile websites may not display correctly.
 - Special characters may display incorrectly.
-

Visual Representation





Meta Tag Syntax

```
<meta attribute="value">
```

Example:

```
<meta charset="UTF-8">
```

Meta tags are self-closing.

Charset Meta Tag

Most important meta tag.

```
<meta charset="UTF-8">
```

Purpose:

Defines character encoding.

UTF-8 supports:

- English
- Hindi
- Arabic
- Chinese
- Emojis

Example:

```
<meta charset="UTF-8">
```

Without UTF-8:

Special characters may display incorrectly.

Viewport Meta Tag

Very important for responsive design.

```
<meta  
name="viewport"  
content="width=device-width, initial-scale=1.0">
```

Purpose:

Makes website responsive on mobile devices.

Without Viewport

Desktop Website



Tiny Mobile Display

With Viewport

Desktop Website



Responsive Mobile Website

Description Meta Tag

Used by search engines.

```
<meta  
name="description"  
content="Learn MERN Stack Development from Beginner to Advanced">
```

Purpose:

Provides page description.

May appear in Google search results.

Example:

MERN Stack Training
Learn MERN Stack Development from Beginner to Advanced

Keywords Meta Tag

```
<meta  
name="keywords"  
content="HTML,CSS,JavaScript,MERN,React">
```

Historically used for SEO.

Modern search engines give very little importance to it.

Favicon

What is a Favicon?

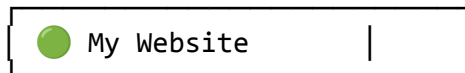
Favicon means:

Favorite Icon

It is the small icon displayed:

- Browser Tab
 - Bookmarks
 - Browser History
 - Mobile Shortcuts
-

Visual Representation



The small icon is called Favicon.

Why Use Favicon?

Benefits:

- Branding
 - Professional Appearance
 - Easy Identification
 - Better User Experience
-

Adding a Favicon

Syntax:

```
<link  
rel="icon"  
href="favicon.ico">
```

Placed inside:

```
<head>  
</head>
```

Example

```
<head>
```

```
<link  
rel="icon"  
href="images/logo.png">
```

```
</head>
```

Common Favicon Formats

ICO

favicon.ico

Traditional format.

PNG

favicon.png

Most commonly used today.

SVG

favicon.svg

Modern scalable format.

Favicon Sizes

Common sizes:

16 x 16
32 x 32
48 x 48
64 x 64

Recommended:

32 x 32

HTML5 Features

What is HTML5?

HTML5 is the latest major version of HTML.

Released:

2014

Developed by:

W3C and WHATWG

Why HTML5 Was Introduced?

Older HTML versions had limitations.

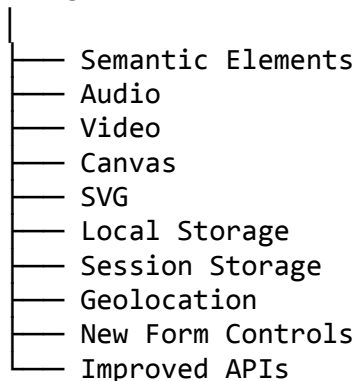
Problems:

- No video support
- No audio support
- Poor semantics
- Limited form controls
- Required plugins

HTML5 solved these issues.

Major HTML5 Features

HTML5



Semantic Elements

Introduced:

`<header>`

`<nav>`


```
<main>
<section>
<article>
<aside>
<footer>
```

Benefits:

- Better SEO
 - Better Accessibility
-

Audio Support

Before HTML5:

Flash required.

Now:

```
<audio controls>

<source src="song.mp3">

</audio>
```

Native browser support.

Video Support

Before HTML5:

Flash Player required.

Now:

```
<video controls>

<source src="video.mp4">

</video>
```

Native support.

Canvas Element

Used for drawing graphics.

Syntax:

```
<canvas
width="300"
```

```
height="200">
</canvas>
```

Applications:

- Games
 - Charts
 - Animations
-

SVG Support

SVG means:

Scalable Vector Graphics

Example:

```
<svg>
```

```
</svg>
```

Benefits:

- Infinite scaling
- Small file size

Used for:

- Logos
 - Icons
 - Diagrams
-

New HTML5 Input Types

Introduced:

```
<input type="email">
```

```
<input type="date">
```

```
<input type="time">
```

```
<input type="url">
```

```
<input type="range">
```

```
<input type="color">
```

Benefits:

- Better validation
 - Better user experience
-

HTML5 Form Validation

Attributes:

required
pattern
maxlength
min
max

Example:

```
<input  
type="email"  
required>
```

Browser validates automatically.

Drag and Drop

HTML5 supports drag and drop.

Example:

```
draggable="true"
```

Applications:

- 22. File Upload
 - 23. Kanban Boards
 - 24. Task Management Apps
-

Offline Web Applications

HTML5 supports offline storage.

Using:

- Local Storage
- Cache Mechanisms

Applications:

- Progressive Web Apps
-

HTML5 vs Older HTML

Feature	HTML4	HTML5
Audio	NO	YES
Video	NO	YES
Canvas	NO	YES
Semantic Tags	NO	YES
Local Storage	NO	YES
Geolocation	NO	YES
New Input Types	NO	YES

Interview Questions

What are Meta Tags?

Meta tags provide information about a webpage to browsers and search engines.

Which Meta Tag Makes Website Responsive?

```
<meta  
name="viewport"  
content="width=device-width, initial-scale=1.0">
```

What is a Favicon?

Small icon displayed on browser tab.

Where is Favicon Added?

Inside:

```
<head>
```

using:

```
<link rel="icon">
```

What is HTML5?

Latest version of HTML introducing multimedia, semantic elements, APIs, and modern web features.

What are Semantic Elements?

Elements that describe content meaning.

Example:

```
<header>  
<nav>  
<footer>
```

Difference Between Local Storage and Session Storage?

Local Storage:

Data remains permanently.

Session Storage:

Data removed when tab closes.

Best Practices

1. Always use UTF-8 charset.
 2. Always include viewport meta tag.
 3. Add meaningful description meta tag.
 4. Use favicon for branding.
 5. Prefer semantic HTML5 elements.
 6. Use HTML5 input types for validation.
 7. Optimize multimedia content.
-

Key Takeaways

Meta Tags:

- charset
- viewport
- description
- author
- robots
- Open Graph

Favicon:

- Browser tab icon
- Branding tool
- Added using link tag

HTML5 Features:

- Semantic Elements

- Audio
- Video
- Canvas
- SVG
- Local Storage
- Session Storage
- Geolocation
- New Input Types
- Better Form Validation

These features make modern websites faster, more accessible, mobile-friendly, SEO-friendly, and interactive.

HTML Lab Projects

Personal Portfolio Website

Problem Statement

You have been hired as a web developer to create your personal portfolio website.

The website should introduce you to visitors and showcase your skills, education, projects, and contact information.

Learning Objectives

After completing this project, students should be able to use:

- HTML Document Structure
- Headings
- Paragraphs
- Images
- Hyperlinks
- Lists
- Tables
- Forms
- Semantic HTML Elements

Requirements

Create the following sections:

Header

Display:

- Your Name
- Professional Title

Example:

Ekta Verma
MERN Stack Developer

About Me

Display:

- Short Introduction
 - Career Objective
-

Profile Picture

Display your image using:

Skills Section

Use an unordered list.

Example:

HTML
CSS
JavaScript
React
Node.js
MongoDB

Education Section

Use a table.

Columns:

- Degree
 - Institute
 - Year
 - Percentage
-

Projects Section

Create a list of projects.

Example:

- Flipkart Clone
 - YouTube Clone
 - AI Resume Analyzer
-

Contact Section

Display:

- Email
- Phone
- LinkedIn
- GitHub

Use hyperlinks where applicable.

Footer

Display:

© 2026 Your Name

Expected Output Structure

HEADER

ABOUT ME

PROFILE IMAGE

SKILLS

EDUCATION TABLE

PROJECTS

CONTACT

FOOTER

Student Registration Form

Problem Statement

Create a Student Registration Form for a training institute.

The form should collect student information and validate important fields using HTML5 validation attributes.

Learning Objectives

Students should learn:

- Forms
 - Input Types
 - Labels
 - Validation
 - Radio Buttons
 - Checkboxes
 - Select Dropdown
 - Textarea
-

Requirements

Collect:

Personal Information

- Full Name
- Email
- Mobile Number

- Date of Birth
-

Gender

Options:

- Male
- Female
- Other

Use radio buttons.

Course Selection

Options:

- MERN Stack
- Java Full Stack
- Python

Use dropdown.

Skills

Options:

- HTML
- CSS
- JavaScript

Use checkboxes.

Address

Use textarea.

Upload Resume

Use file input.

Buttons

- Submit
- Reset

Expected Output

Student Registration Form

Name

Email

Phone

DOB

Gender

Course

Skills

Address

Resume Upload

Submit

Reset

Student Registration Form

Personal Information

Full Name: *

Email: *

Mobile Number: *

Date of Birth: 

Gender: ☐ Male ☐ Female ☐ Other *

Course Information

Course: ▼ *

Skills (Select all that apply)

- ☐ HTML
- ☐ CSS
- ☐ JavaScript
- ☐ React
- ☐ Node.js

Address

Address: *

Upload Resume

Resume: No file chosen *

Additional Information

How did you hear about us? ▼

Any Message:

© 2026 Training Institute. All Rights Reserved.

Website Layout Using Semantic HTML

Problem Statement

Create a simple news website layout using Semantic HTML elements.

The objective is to learn how modern websites structure content.

Use only HTML.

No CSS.

Learning Objectives

Students should learn:

- header
 - nav
 - main
 - section
 - article
 - aside
 - footer
-

Requirements

Header

Website Name

Navigation

Links:

- Home
 - News
 - Sports
 - Contact
-

Main Content

Create two news articles.

Sidebar

Display:

- Latest News
 - Advertisements
-

Footer

Visual Layout

Daily News

Latest Updates From Around The World

Home | News | Sports | Contact

Technology News

AI is transforming software development.

Sports News

India wins the latest cricket series.

Latest Updates

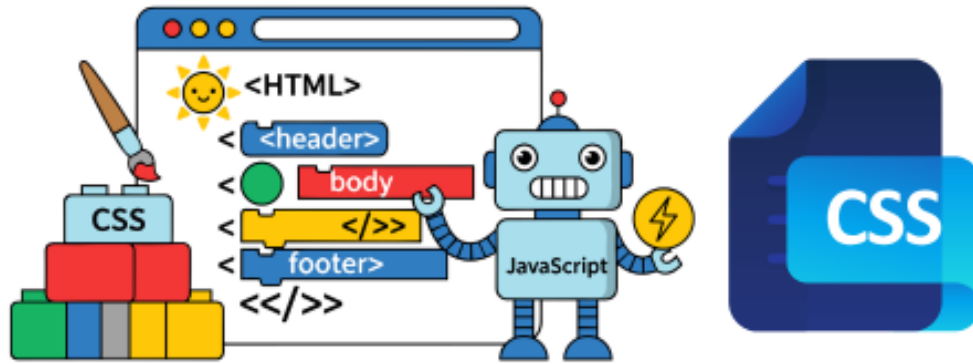
- MERN Course Started
 - New AI Workshop
-

© 2026 Daily News

After All above tasks Completion

After completing all projects, add the following:

1. Add favicon to all projects.
2. Add meta tags.
3. Add images.
4. Add tables where applicable.
5. Create navigation links between pages.
6. Add HTML5 validation.
7. Add semantic elements wherever possible.



CSS Notes

Introduction to CSS

What is CSS?

CSS stands for:

Cascading Style Sheets

CSS is used to control the appearance and presentation of HTML elements.

HTML creates the structure of a webpage.

CSS makes the webpage attractive.

Example:

Without CSS:

```
<h1>Welcome to MERN Training</h1>
```

Output:

Simple black heading.

With CSS:

```
<h1 style="color:red;">
  Welcome to MERN Training
</h1>
```

Output:

Red colored heading.

Real Life Example

Think about building a house.

HTML = Structure of House

30. Walls

31. Doors

32. Windows

33. Rooms

CSS = Decoration

- Paint
- Furniture
- Curtains
- Lighting

JavaScript = Functionality

- Automatic Door
 - Smart Lights
 - Security System
-

Why CSS?

Without CSS:

- Websites look plain
- Poor user experience

With CSS:

- Attractive design
 - Better readability
 - Responsive layouts
 - Better user engagement
-

Difference Between HTML, CSS and JavaScript

HTML	CSS	JavaScript
Structure	Styling	Functionality
Skeleton	Appearance	Behavior
Content	Design	Interaction
Static	Visual	Dynamic

Example

HTML

```
<button>Click Me</button>
```


CSS

```
button{  
    background-color: blue;  
    color: white;  
}
```

JavaScript

```
button.addEventListener("click",()=>{  
    alert("Button Clicked");  
});
```

Types of CSS

There are 3 ways to apply CSS.

```
CSS  
├── Inline CSS  
├── Internal CSS  
└── External CSS
```

Inline CSS

CSS written directly inside HTML element.

Syntax:

```
<tag style="property:value;">
```

Example:

```
<h1 style="color:red;">  
    Welcome  
</h1>
```

Output:

Red heading.

Advantages

- Quick testing
 - Easy for small changes
-

Disadvantages

- Difficult to maintain

- Repeated code
 - Not scalable
-

Internal CSS

Written inside:

```
<style>  
</style>
```

tag.

Example:

```
<style>  
  
h1{  
    color:red;  
}  
  
</style>
```

External CSS

Separate file.

style.css

```
h1{  
    color:red;  
}
```

HTML

```
<link rel="stylesheet" href="style.css">
```

Most commonly used in industry.

Color Property

Used to change text color.

Syntax:

```
color: value;
```

Example:

```
color: red;
```

Ways to Specify Colors

CSS provides multiple ways.

1. Color Name

color:*red*;

color:*blue*;

color:*green*;

Advantages:

Easy to understand.

2. HEX Code

HEX = Hexadecimal

Starts with #

Example:

color:*#ff0000*;

Red

color:*#0000ff*;

Blue

color:*#00ff00*;

Green

Structure

#RRGGBB

Example:

#FF0000

FF = Red

00 = Green

00 = Blue

3. RGB Values

RGB means:

Red Green Blue

Example:

```
color: rgb(255,0,0);
```

Red

```
color: rgb(0,255,0);
```

Green

```
color: rgb(0,0,255);
```

Blue

Comparison

Color	Name	HEX	RGB
Red	red	#ff0000	rgb(255,0,0)
Green	green	#00ff00	rgb(0,255,0)
Blue	blue	#0000ff	rgb(0,0,255)

Background Color

Changes background of element.

Syntax:

```
background-color: value;
```

Example:

```
background-color: yellow;
```

Example

```
<p style="
color:white;
background-color:black;
">
Hello CSS
</p>
```

Why Do We Use px?

px stands for:

Pixel

A pixel is the smallest unit of display on a screen.

Example:

```
font-size:20px;
```

Means:

20 pixels tall.

Why Pixels?

- Precise control
 - Consistent sizing
 - Easy to understand
-

Common Pixel Values

```
font-size:12px;
```

Small text

```
font-size:16px;
```

Normal text

```
font-size:24px;
```

Heading

```
font-size:48px;
```

Large heading

Font Size Property

Controls text size.

Syntax:

```
font-size: value;
```

Example:

```
font-size:30px;
```

Visual Representation

12px Small

16px Normal

24px Medium

48px Large

Font Weight Property

Controls thickness of text.

Syntax:

```
font-weight:value;
```

Values

Normal

```
font-weight:normal;
```

Bold

```
font-weight:bold;
```

Numeric Values

```
font-weight:100;
```

Thin

```
font-weight:400;
```

Normal

```
font-weight:700;
```

Bold

```
font-weight:900;
```

Extra Bold

Visual Representation

100 Thin

300 Light

400 Normal

700 Bold

900 Extra Bold

Text Align Property

Controls horizontal alignment.

Syntax:

```
text-align: value;
```

Left

```
text-align: left;
```

Default

Center

```
text-align: center;
```

Right

```
text-align: right;
```

Justify

```
text-align: justify;
```

Text spreads evenly.

Example

```
text-align: center;
```

Output:

Centered text.

Block and Inline Elements

Understanding this concept is critical.

Block Elements

Take full width.

Always start on new line.

Examples:

```
<div>
<p>
<h1>
<section>
```

Visual

```
+-----+
| Block Element |
+-----+
```

Example

```
<p>HTML</p>
```

```
<p>CSS</p>
```

Output:

HTML

CSS

Each starts on new line.

Inline Elements

Take only required width.

Do not start new line.

Examples:

```
<span>
<a>
<strong>
<em>
```

Visual

HTML CSS JavaScript

All remain in same line.

Example

```
<span>HTML</span>
```

```
<span>CSS</span>
```

Output:

HTML CSS

Display Property

Changes default behavior.

Syntax:

```
display: value;
```

display: block

Makes element block-level.

Example:

```
display: block;
```

Visual:

Element
(New Line)

display: inline

Makes element inline.

Example:

```
display: inline;
```

Visual:

Element Element

display:inline-block

Combination of both.

Features:

- Remains inline
- Allows width and height

Example:

display:*inline-block*;

Used heavily in menus and cards.

Comparison

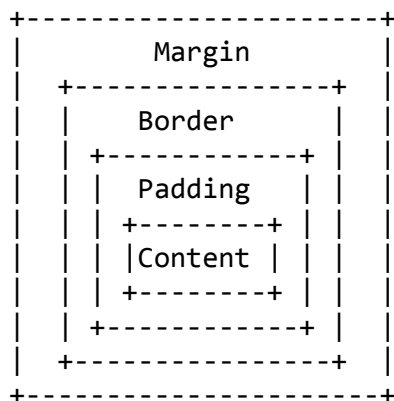
Property	New Line	Width/Height
block	Yes	Yes
inline	No	No
inline-block	No	Yes

CSS Box Model

Most important CSS concept.

Every HTML element is a rectangular box.

Visual Representation



Content

Actual text or image.

Example:

```
<p>Hello World</p>
```

Hello World = Content

Padding

Space inside the border.

```
padding:20px;
```

Visual

Border

Padding

Content

Border

Boundary around element.

Syntax:

```
border:1px solid black;
```

Components:

Width

Style

Color

Example:

```
border:2px solid red;
```

Margin

Space outside border.

```
margin:20px;
```

Visual

Element

Margin

Other Element

Example

`padding:20px;`

`border:2px solid black;`

`margin:30px;`

Shorthand Properties

Padding

`padding:20px;`

All sides.

Margin

`margin:20px;`

All sides.

Individual Sides

`margin-top:20px;`

`margin-right:10px;`

`margin-bottom:20px;`

`margin-left:10px;`

Padding Individual

`padding-top:20px;`

`padding-right:10px;`

`padding-bottom:20px;`

`padding-left:10px;`

Interview Questions

What is CSS?

CSS stands for Cascading Style Sheets and is used to style HTML elements.

Difference Between HTML, CSS and JavaScript?

HTML = Structure

CSS = Styling

JavaScript = Functionality

What are the Types of CSS?

- Inline
 - Internal
 - External
-

Difference Between Block and Inline Elements?

Block:

Starts on new line.

Inline:

Stays in same line.

Difference Between Margin and Padding?

Margin:

Outside spacing.

Padding:

Inside spacing.

What is Box Model?

Content + Padding + Border + Margin

Difference Between display:block and display:inline?

Block takes full width.

Inline takes only required width.

Why is inline-block useful?

Allows elements to stay inline while supporting width and height.

Key Takeaways

- CSS controls presentation of HTML.
 - Three types of CSS: Inline, Internal, External.
 - Colors can be specified using Name, HEX, RGB.
 - px provides precise sizing.
 - font-size controls size.
 - font-weight controls thickness.
 - text-align controls alignment.
 - Block elements occupy full width.
 - Inline elements occupy required width.
 - Display property changes layout behavior.
 - Every element follows the Box Model: Content → Padding → Border → Margin.
-

CSS Selectors:

Before Learning Selectors

Students often ask:

Why do we need selectors?

Answer:

Because selectors help us move from **Inline CSS** to **Internal CSS** and **External CSS**, which is how CSS is actually written in real projects.

Problem with Inline CSS

Suppose we have 3 headings.

```
<h1 style="color:red;">HTML</h1>
```

```
<h1 style="color:red;">CSS</h1>
```

```
<h1 style="color:red;">JavaScript</h1>
```

Output:

All headings become red.

Looks fine.

But imagine a website with:

100 headings
50 paragraphs
30 buttons
20 cards

Now you must write:

```
style="color:red"
```

again and again.

This creates:

- 34. Duplicate code
 - 35. Difficult maintenance
 - 36. Poor readability
 - 37. Not scalable
-

Real Industry Example

Imagine Flipkart has:

10,000 Product Titles

Would developers write?

```
<h2 style="color:blue;">iPhone</h2>
```

```
<h2 style="color:blue;">Samsung</h2>
```

```
<h2 style="color:blue;">OnePlus</h2>
```

...

No.

That would be a disaster.

Solution: Internal CSS

Instead of writing style repeatedly:

```
<style>
```

```
h2{  
  color:blue;
```

```
}
```

```
</style>
```

Now:

```
<h2>iPhone</h2>  
<h2>Samsung</h2>  
<h2>OnePlus</h2>
```

All headings become blue automatically.

How Browser Understands This?

Browser asks:

Find all h2 tags
Apply color blue

The process of finding elements is called:

SELECTOR

What is a Selector?

A Selector is a mechanism used to select HTML elements so CSS can be applied to them.

Simple definition:

Selector tells CSS WHERE to apply styling.

CSS Selectors

Selectors

- Universal Selector
 - Tag Selector
 - Attribute Selector
 - ID Selector
 - Class Selector
 - Group Selector
-

1. Universal Selector (*)

Definition

Selects every element on the page.

Syntax:

```
*{  
  property:value;  
}
```

Example:

```
<h1>HTML</h1>
```

```
<p>CSS</p>
```

```
<button>Submit</button>
```

```
*{  
  color:red;  
}
```

Output:

Everything becomes red.

Visual Representation

```
*  
├── h1  
├── p  
├── button  
├── div  
└── span
```

Everything selected.

Industry Usage

Most common use:

```
*{  
  margin:0;  
  padding:0;  
  box-sizing:border-box;  
}
```

Used in almost every project.

2. Tag Selector

Also called:

Element Selector

Selects all occurrences of a tag.

Example

HTML

```
<h1>Welcome</h1>
```

```
<h1>Learning CSS</h1>
```

```
<p>Paragraph</p>
```

CSS

```
h1{  
  color:red;  
}
```

Output:

Only h1 becomes red.

Paragraph remains unchanged.

Visual Representation

```
h1  
├──  
└── Welcome  
    Learning CSS
```

Another Example

```
p{  
  font-size:20px;  
}
```

All paragraphs get font size 20px.

Real World Example

```
button{  
  background-color:blue;
```

```
color:white;
}
```

Every button receives same styling.

3. ID Selector

Used to select a unique element.

Syntax:

```
#idname{
}
```

Example

HTML

```
<h1 id="title">
Welcome To MERN
</h1>
```

CSS

```
#title{
color:red;
}
```

Output:

Only title becomes red.

Visual Representation

```
#title
  |
  ▼
Welcome To MERN
```

Rule

ID should be unique.

Good:

```
<h1 id="header">
```

Bad:

```
<h1 id="header">
<p id="header">
```

Duplicate ID.

Real World Example

```
<section id="hero">

</section>

#hero{
  background-color:black;
}
```

4. Class Selector

Most important selector in industry.

Used to style multiple elements.

Syntax:

```
.classname{
}
```

Example

HTML

```
<p class="text">
HTML
</p>
```

```
<p class="text">
CSS
</p>
```

```
<p class="text">
JavaScript
</p>
```

CSS

```
.text{
  color:blue;
}
```

Output:

All paragraphs become blue.

Visual Representation

```
.text
{
  — HTML
  — CSS
  — JavaScript
```

Why Class?

Because styling is reusable.

Imagine:

50 buttons

Instead of:

```
#btn1
#btn2
#btn3
```

Use:

```
.btn{
  background-color:blue;
}
```

All buttons share styling.

Real Industry Example

HTML

```
<button class="btn">
Login
</button>
```

```
<button class="btn">
Register
</button>
```

CSS

```
.btn{
  background-color:green;
  color:white;
}
```

Difference Between ID and Class

ID	Class
Unique	Reusable
#	.
One Element	Multiple Elements
Higher Priority	Lower Priority

5. Attribute Selector

Selects elements based on attributes.

Example

HTML

```
<input type="text">
```

```
<input type="password">
```

CSS

```
input[type="text"]{  
  background-color:yellow;  
}
```

Output:

Only text input becomes yellow.

Visual Representation

```
input  
├── type=text  
└── type=password
```

Real World Example

Style all email fields.

```
input[type="email"]{  
  border:2px solid green;  
}
```

Another Example

HTML

```
<a href="#" target="_blank">  
Google  
</a>
```

CSS

```
a[target="_blank"]{  
  color:red;  
}
```

Only links opening in new tab become red.

6. Group Selector

Used when same CSS should apply to multiple selectors.

Without Group Selector

```
h1{  
  color:red;  
}
```

```
p{  
  color:red;  
}
```

```
button{  
  color:red;  
}
```

Repetition.

With Group Selector

```
h1,p,button{  
  color:red;  
}
```

Cleaner code.

Visual Representation

```
h1  
p
```

button
↓
color:red

Real World Example

```
h1, h2, h3, h4, h5, h6{  
  font-family:Arial;  
}
```

All headings use same font.

Selector Priority (Specificity)

Suppose:

```
<p  
id="title"  
class="text">  
Hello  
</p>
```

CSS

```
p{  
  color:blue;  
}
```

```
.text{  
  color:green;  
}
```

```
#title{  
  color:red;  
}
```

Output:

Red

Because:

ID > Class > Tag

Priority:

Inline CSS

↓

ID

↓

Class

↓

Tag

↓
Universal

Selector	Symbol	Example
Universal	*	*{}
Tag	tagname	h1{}
ID	#	#title{}
Class	.	.btn{}
Attribute	[]	inputtype="text"
Group	,	h1,p,button{}

Important Interview Question

Which selector is used most in real projects?

Class Selector

Because it is reusable, maintainable, and works perfectly with modern frameworks like React, Angular, Vue, Next.js, and MERN applications.

CSS Flexbox -(Parent & Child Properties)

Introduction to Flexbox

Before Flexbox, creating layouts was difficult.

Developers used:

38. Tables

39. Float

40. Position

41. Inline-block

Problems:

Complex code

Alignment issues

Responsive design difficult

Vertical centering difficult

What is Flexbox?

Flexbox stands for:

Flexible Box Layout

It is a one-dimensional layout system used to arrange items in:

- Row (Horizontal)
 - Column (Vertical)
-

Real Life Example

Imagine a table with 4 water bottles.

Normal HTML:

Bottle
Bottle
Bottle
Bottle

One below another.

Flexbox:

Bottle Bottle Bottle Bottle

Arranged in a row.

Why Flexbox?

Flexbox solves:

- Horizontal alignment
 - Vertical alignment
 - Equal spacing
 - Responsive layouts
 - Dynamic sizing
-

Most Important Concept

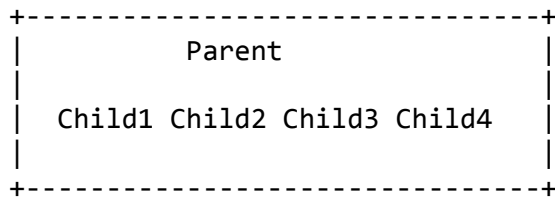
Flexbox always has:

Flex Container (Parent)



Flex Items (Children)

Visual:



Step 1: Make Parent a Flex Container

HTML

```
<div class="container">

    <div>1</div>
    <div>2</div>
    <div>3</div>

</div>
```

CSS

```
.container{
    display: flex;
}
```

Output:

1 2 3

Without flex:

1
2
3

Parent Properties (Flex Container)

display
flex-direction
justify-content
align-items
flex-wrap
align-content
gap
row-gap
column-gap

PARENT PROPERTY 1: display: flex

Converts parent into flex container.

```
.container{  
  display: flex;  
}
```

Visual

Without Flex

Box1
Box2
Box3

With Flex

Box1 Box2 Box3

PARENT PROPERTY 2: flex-direction

Controls direction of items.

Values:

row
row-reverse
column
column-reverse

row (Default)

flex-direction: *row*;

Output:

1 2 3

row-reverse

flex-direction: *row-reverse*;

Output:

3 2 1

column

flex-direction: *column*;

Output:

1
2
3

column-reverse

flex-direction: *column-reverse*;

Output:

3
2
1

Visual Diagram

ROW

1 2 3

COLUMN

1
2
3

PARENT PROPERTY 3: justify-content

Controls alignment on Main Axis.

Most important property.

Values:

flex-start
center
flex-end
space-between
space-around
space-evenly

flex-start

Default

`justify-content: flex-start;`

1 2 3

center

`justify-content: center;`

1 2 3

flex-end

`justify-content: flex-end;`

1 2 3

space-between

`justify-content: space-between;`

|1 ===== 2 ===== 3|

First item touches the left edge.

Last item touches the right edge.

Only the space **between** items is equal.

space-around

`justify-content: space-around;`

|----1-----2-----3----|

Each item gets **equal space** around itself.

Since neighboring items each contribute half of their surrounding space, the gap **between two items is twice the edge gap**.

space-evenly

`justify-content: space-evenly;`

|-----1-----2-----3-----|

Every empty gap is **exactly the same size**.

Industry Example

Navigation Menu

```
nav{  
  display:flex;  
  justify-content:space-between;  
}
```

Output:

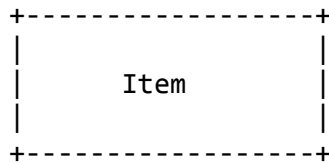
Logo

Menu

center

align-items:*center*;

Visual:



Perfect vertical center.

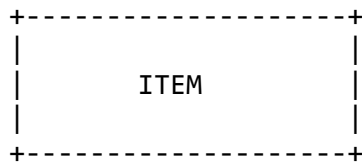
Most Important Interview Question

```
Justify-content:center; //rows  
Align-items:center;    //columns
```

Creates:

Perfect Center Alignment

Visual



PARENT PROPERTY 5: flex-wrap

Default:

`flex-wrap: nowrap;`

All items stay in one line.

Problem

1 2 3 4 5 6 7 8 9

May overflow.

wrap

`flex-wrap: wrap;`

Output:

1 2 3 4

5 6 7 8

9

Items move next line.

Industry Example

Product Cards

```
.products{  
  display: flex;  
  flex-wrap: wrap;  
}
```

PARENT PROPERTY 6: gap

Adds spacing between items.

`gap: 20px;`

Visual:

1 2 3

row-gap

`row-gap:20px;`

column-gap

`column-gap:20px;`

Interview Questions

What is Flexbox?

A one-dimensional layout system used to arrange items in rows or columns.

Difference Between justify-content and align-items?

justify-content:

Works on horizontal items .

align-items:

Works on vertical items.

Why use flex-wrap?

Allows items to move to next line when space is insufficient.

What does flex:1 mean?

Item takes available remaining space.

Flexbox Learning Formula

STEP 1

`display:flex`

STEP 2

`flex-direction`

STEP 3

justify-content

STEP 4

align-items

STEP 5

flex-wrap

STEP 6

gap

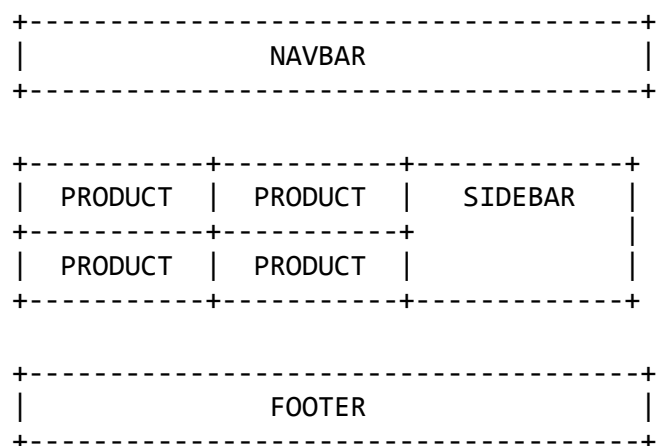
Master these 6 properties and you can build nearly 90% of layouts used in modern React, Angular, Vue, Next.js, and MERN Stack applications.

CSS Grid Layout

Before Learning CSS Grid

Let's understand a problem first.

Suppose you want to create a webpage like this:



Question:

Can we build this using HTML?

Yes

Can we build this using Flexbox?

Yes

Is Flexbox the best solution?

No

Because Flexbox is designed for:

Row Layouts

OR

Column Layouts

But not both together.

Problem With Flexbox

Flexbox is:

One Dimensional

Meaning:

ROW

OR

COLUMN

Visual:

1 2 3 4 5

OR

1

2

3

4

5

But modern websites require:

Rows + Columns Together

Example:

Dashboard

Admin Panel

LMS

CRM

Analytics System

Portfolio Website

This is where CSS Grid comes in.

What is CSS Grid?

CSS Grid is a:

Two Dimensional Layout System

It works with:

Rows

AND

Columns

at the same time.

Real Life Example

Imagine a Chess Board.

```
+---+---+---+---+
|   |   |   |   |
+---+---+---+---+
|   |   |   |   |
+---+---+---+---+
|   |   |   |   |
+---+---+---+---+
|   |   |   |   |
+---+---+---+---+
```

Chess Board contains:

Rows

Columns

Cells

CSS Grid works exactly like this.

Grid Container

Parent element.

HTML

```
<div class="container">
```

```
<div>1</div>
```

```
<div>2</div>
```

```
<div>3</div>
```

```
</div>
```

CSS

```
.container{  
  display:grid;  
}
```

Parent becomes:

Grid Container

Grid Items

Children of grid container.

Example:

```
<div>1</div>
```

```
<div>2</div>
```

```
<div>3</div>
```

These are Grid Items.

Most Important Concept: Grid Lines

The moment browser sees:

```
display:grid;
```

it creates invisible lines.

Visual:

	Column Lines			
	1	2	3	4
1	-----	-----	-----	
2	-----	-----	-----	
3	-----	-----	-----	
4	-----	-----	-----	

These numbers are called:

Grid Lines

Everything in the Grid works using these lines.

Grid Cells

Smallest unit inside Grid.

Visual:

```
+-----+  
|Cell  |  
+-----+
```

Every box is called a Cell.

Grid Tracks

Distance between two lines.

Example:

Line 1 ----- Line 2

This distance is called:

Track

Parent Properties of Grid

Grid properties are divided into:

Parent Properties

Child Properties

Parent Properties

`display:grid`

`grid-template-columns`

`grid-template-rows`

`gap`

`row-gap`

column-gap

justify-items

align-items

justify-content

align-content

grid-template-areas

Child Properties

grid-column

grid-row

grid-area

justify-self

align-self

Property 1: display:grid

Converts parent into Grid Container.

Example:

```
.container{  
  display:grid;  
}
```

Without Grid:

1
2
3
4

With Grid + Columns:

1 2 3 4

Property 2: grid-template-columns

Used to create columns.

Syntax:

```
grid-template-columns:
100px 100px 100px;
```

Visual:

```
+-----+-----+-----+
|       |       |       |
+-----+-----+-----+
```

3 columns created.

Example

```
.container{

  display:grid;

  grid-template-columns:
  100px 100px 100px;
}
```

Output

```
+-----+-----+-----+
|  1  |  2  |  3  |
+-----+-----+-----+
```

Different Width Columns

```
grid-template-columns:
100px 200px 300px;
```

Visual:

```
+---+-----+-----+
|100| 200 |   300   |
+---+-----+-----+
```

Property 3: grid-template-rows

Creates rows.

Example:

```
grid-template-rows:
100px 100px 100px;
```

Visual:


```
+-----+
|       |
+-----+
```

```
+-----+
|       |
+-----+
```

```
+-----+
|       |
+-----+
```

3 rows created.

Example

```
.container{
  display:grid;

  grid-template-columns:
    100px 100px;

  grid-template-rows:
    100px 100px;
}
```

Visual:

```
+-----+-----+
|  1  |  2  |
+-----+-----+
```

```
|  3  |  4  |
+-----+-----+
```

Why Rows and Columns Together?

Flexbox:

Row OR Column

Grid:

Rows AND Columns

This is Grid's biggest superpower.

Property 4: gap

Adds spacing between rows and columns.

Without Gap:

```
+---+ +---+ +---+
| 1 | | 2 | | 3 |
+---+ +---+ +---+
```

With Gap:

`gap:20px;`

Visual:

```
+---+   +---+   +---+
| 1 |   | 2 |   | 3 |
+---+   +---+   +---+
```

Property 5: row-gap

Adds vertical spacing only.

`row-gap:20px;`

Property 6: column-gap

Adds horizontal spacing only.

`column-gap:20px;`

Quick Summary

`display:grid`

Creates Grid Container

`grid-template-columns`

Creates Columns

`grid-template-rows`

Creates Rows

`gap`

Spacing Between Rows and Columns

row-gap

Vertical Space

column-gap

Horizontal Space

CSS Position Property

Real Life Example

Imagine students sitting in a classroom.

Rahul

Aisha

Priya

Rohit

Everyone is sitting normally.

This is:

position: static;

Teacher says:

"Ekta move slightly right."

This is:

position: relative;

Teacher says:

"Stand at the corner of the classroom."

This is:

position: absolute;

Teacher says:

"Follow me everywhere."

This is:

position: fixed;

Teacher says:

"Stay at top when class scrolls."

This is:

position: sticky;

Types of Position

Position

|

|— static

|— relative

|— absolute

|— fixed

|— sticky

Position Coordinates

Position works with:

top

right

bottom

left

Visual:

TOP



LEFT ← Element → RIGHT



BOTTOM

These properties tell browser where to move an element.

Position: Static

Default position of every HTML element.

position: static;

Example

```
<div>HTML</div>
```

```
<div>CSS</div>
```

```
<div>JavaScript</div>
```

Output:

HTML

CSS

JavaScript

Important Rule

Static ignores:

top

right

bottom

left

Example:

```
div{  
  position:static;  
  top:100px;  
}
```

Nothing happens.

Because static elements cannot be moved using coordinates.

Position: Relative

Most important concept.

What Does Relative Mean?

Relative means:

Move relative to original position.

The element keeps its original space.

Example

```
<div class="box">
```

BOX

</div>

```
.box{  
  position:relative;  
  left:50px;  
}
```

Before

BOX

After

BOX

Moved right.

Understanding top, right, bottom, left with Relative

```
.box{  
  position:relative;  
  top:50px;  
}
```

Meaning:

Move DOWN by 50px.

```
.box{  
  position:relative;
```

```
    left:50px;  
}
```

Meaning:

Move RIGHT by 50px.

```
.box{  
    position:relative;  
    right:50px;  
}
```

Meaning:

Move LEFT by 50px.

```
.box{  
    position:relative;  
    bottom:50px;  
}
```

Meaning:

Move UP by 50px.

Why Is Relative Important?

Because:

Absolute Depends on Relative

What Does Absolute Mean?

Absolute means:

Remove elements from normal document flow.

Example -

HTML

```
<div class="red"></div>
```

```
<div class="blue"></div>
```

CSS

```
div{  
    width:100px;  
    height:100px;  
}  
.red{  
    background:red;  
}  
.blue{  
    background:blue;  
}
```

Both divs are **block elements**.

Block elements naturally appear one below another.

Page

```
+-----+  
| RED   |  
+-----+  
+-----+  
| BLUE  |  
+-----+
```

This is called **Normal Flow**.

Now add **position:absolute**

```
.red{  
  background:red;  
  position:absolute;  
}
```

The browser says:

"The red box is absolute."

"I will NOT reserve space for it." - IGNORE

So the blue box moves up.

Normal flow becomes

+-----+

| BLUE |

+-----+

The red box is **not gone**.

It is simply floating separately.

Text on Images

Suppose you have this.

+-----+

| |

| Mountains |

| |

+-----+

You want

+-----+

```
| Amazing View |  
|               |  
|   Mountains   |  
|               |  
+-----+
```

The text should sit **over the image**.

```
.image{  
  position:relative;  
}
```

```
.text{  
  position:absolute;  
  top:20px;  
  left:20px;  
}
```

The text is placed on top without affecting the image size.

Position: Fixed

Fixed means:

Attach element to browser window.

Example

```
.chat{  
  position:fixed;  
  bottom:20px;  
  right:20px;  
}
```

Visual:

```
+-----+
```



The element never moves.

Industry Examples

WhatsApp Button



Back To Top Button



Support Widget

Help

Call Button



Position: Sticky

Example

```
header{  
  position:sticky;  
  top:0;  
}
```

Before Scroll

HEADER

CONTENT

After Scroll

HEADER

(stays visible)

CONTENT

Industry Examples

- Amazon Navigation
 - News Website Header
 - Documentation Sidebar
 - Course Navigation
 - Product Category Menu
-

Sticky vs Fixed

Fixed

Always fixed.



Never moves.

CSS Transition - Complete Deep Dive Notes

Before Learning Transition

Imagine a button.

HTML:

```
<button>Click Me</button>
```

CSS:

```
button{  
    background-color:blue;  
}
```

```
button:hover{  
    background-color:red;  
}
```

Output:

Normal:

BLUE BUTTON

Hover:

RED BUTTON

Question:

What happens?

Color changes instantly.

There is no animation.

It feels robotic.

Transition allows CSS property values to change smoothly over a period of time.

Without Transition:

Instant Change

With Transition:

Smooth Change

Transition Properties

Transition has 4 major properties.

transition-property

transition-duration

transition-timing-function

transition-delay

1. transition-property

Determines which property should animate.

Example:

```
transition-property:background-color;
```

Meaning:

Animate only background color

Example

```
button{  
  
  background:blue;  
  
  transition-property:background-color;  
  
  transition-duration:2s;
```

```
}
```

```
button:hover{  
  background:red;  
}
```

2. transition-duration

Controls speed.

Example:

```
transition-duration:2s;
```

Meaning: Animation takes 2 seconds

Example

```
transition-duration:5s;
```

Units

1s

2s

3s

OR

500ms

1000ms

2000ms

3. transition-delay

Controls waiting time.

Example:

```
transition-delay:2s;
```

Meaning:

Wait 2 seconds

Then start animation

Example

```
button{  
  transition:2s;  
  transition-delay:3s;  
}
```

4. transition-timing-function

The `transition-timing-function` property specifies the speed curve of the transition effect.

This property can have one of the following values:

- **ease** - transition will start slow, then go fast, and end slow (this is default)
- **linear** - transition will keep the same speed from start to end
- **ease-in** - transition will start slow
- **ease-out** - transition will end slow
- **ease-in-out** - transition will have a slow start and end

CSS Animation - Complete Deep Dive Notes (Corporate Trainer Edition)

Animation vs Transition

Transition

Requires Event, Hover, Focus, Click

Example:

```
button:hover{  
  background:red;  
}
```

Animation

Runs Automatically

Example:

```
animation:  
spin 2s infinite;
```

An animation lets an element gradually change from one style to another.

To use CSS animation, you must specify some keyframes for the animation.

Keyframes hold what styles the element will have at certain times.

Animation Components

CSS Animation has two parts.

1. @keyframes
2. animation properties

CSS @keyframes Rule

When you specify CSS styles inside the @keyframes rule, the animation will gradually change from the current style to the new style at certain times.

To get an animation to work, you must bind the animation to an element.

The following example binds the "myAnimation" animation to the <div> element. The animation will last for 4 seconds, and it will gradually change the background-color of the <div> element from "red" to "yellow":

```
/* The animation code */

@keyframes myAnimation {

  from {background-color: red;}

  to {background-color: yellow;}

}

/* The element to apply the animation to */

div {

  background-color: red;

  animation-name: myAnimation;

  animation-duration: 4s;

}
```

In the example above we have used the keywords "from" and "to" in the @keyframes rule, which represents 0% (start) and 100% (complete).

It is also possible to use percent. By using percent, you can add as many style changes as you like.

The following example will change the background-color of the <div> element when the animation is 25% complete, 50% complete, and again when the animation is 100% complete:

```
@keyframes myAnimation {

  0% {background-color: red;}

  25% {background-color: yellow;}

  50% {background-color: blue;}

  100% {background-color: green;}

}

div {

  background-color: red;
```

```
animation-name: myAnimation;  
animation-duration: 4s;  
}
```

The following example will change both the background-color and the position of the <div> element when the animation is 25% complete, 50% complete, and again when the animation is 100% complete:

```
@keyframes myAnimation {  
  0% {background-color:red; left:0px; top:0px;}  
  25% {background-color:yellow; left:200px; top:0px;}  
  50% {background-color:blue; left:200px; top:200px;}  
  75% {background-color:green; left:0px; top:200px;}  
  100% {background-color:red; left:0px; top:0px;}  
}  
  
div {  
  background-color: red;  
  animation-name: myAnimation;  
  animation-duration: 4s;  
}
```

CSS animation-duration Property

Controls speed.

Example:

```
animation-duration:3s;
```

Meaning:

Animation takes 3 seconds

CSS animation-delay Property

The `animation-delay` property specifies a delay for the start of an animation.

The following example has a 2 seconds delay before starting the animation:

```
div {  
    background-color: red;  
    animation-name: myAnimation;  
    animation-duration: 4s;  
    animation-delay: 2s;  
}
```

Negative values are also allowed. If using negative values, the animation will start as if it had already been playing for *N* seconds.

In the following example, the animation will start as if it had already been playing for 2 seconds:

```
div {  
    background-color: red;  
    animation-name: myAnimation;  
    animation-duration: 4s;  
    animation-delay: -2s;  
}
```

CSS animation-iteration-count Property

The `animation-iteration-count` property specifies the number of times an animation should run.

The following example will run the animation 3 times before it stops:

```
div {  
    background-color: red;  
    animation-name: myAnimation;  
    animation-duration: 4s;  
    animation-iteration-count: 3;  
}
```

The following example uses the value "infinite" to make the animation continue for ever:

```
div {
```

```
background-color: red;

animation-name: myAnimation;

animation-duration: 4s;

animation-iteration-count: infinite;

}
```

CSS animation-timing-function Property

The `animation-timing-function` property specifies the speed curve of the animation.

The `animation-timing-function` property can have the following values:

- `ease` - Specifies an animation with a slow start, then fast, then end slowly (this is default)
- `linear` - Specifies an animation with the same speed from start to end
- `ease-in` - Specifies an animation with a slow start
- `ease-out` - Specifies an animation with a slow end
- `ease-in-out` - Specifies an animation with a slow start and end

The following example shows some of the different speed curves that can be used:

```
#div1 {animation-timing-function: linear;}
```

```
#div2 {animation-timing-function: ease;}
```

```
#div3 {animation-timing-function: ease-in;}
```

```
#div4 {animation-timing-function: ease-out;}
```

```
#div5 {animation-timing-function: ease-in-out;}
```

CSS animation-direction Property

The `animation-direction` property specifies whether an animation should be played forwards, backwards or in alternate cycles.

The `animation-direction` property can have the following values:

- `normal` - The animation is played as normal (forwards). This is default

- **reverse** - The animation is played in reverse direction (backwards)
- **alternate** - The animation is played forwards first, then backwards
- **alternate-reverse** - The animation is played backwards first, then forwards

CSS animation-fill-mode Property

CSS animations do not affect an element before the first keyframe is played or after the last keyframe is played. The animation-fill-mode property can override this behavior.

The animation-fill-mode property specifies a style for the target element when the animation is not playing (before it starts, after it ends, or both).

The animation-fill-mode property can have the following values:

- **none** - Default value. Animation will not apply any styles to the element before or after it is executing
- **forwards** - The element will retain the style values that is set by the last keyframe (depends on animation-direction and animation-iteration-count)
- **backwards** - The element will get the style values that is set by the first keyframe (depends on animation-direction), and retain this during the animation-delay period
- **both** - The animation will follow the rules for both forwards and backwards, extending the animation properties in both directions

The following example lets the <div> element retain the style values from the last keyframe when the animation ends:

```
div {  
  
    background: red;  
  
    animation-name: myAnimation;  
  
    animation-duration: 3s;  
  
    animation-fill-mode: forwards;  
  
}
```

The following example lets the <div> element get the style values set by the first keyframe before the animation starts (during the animation-delay period):

```
div {  
  
    background: red;  
  
    animation-name: myAnimation;  
  
    animation-duration: 3s;  
  
    animation-delay: 2s;  
  
    animation-fill-mode: backwards;  
  
}
```

The following example lets the <div> element get the style values set by the first keyframe before the animation starts, and retain the style values from the last keyframe when the animation ends:

```
div {  
  
    width: 100px;  
  
    height: 100px;  
  
    background: red;  
  
    position: relative;  
  
    animation-name: myAnimation;  
  
    animation-duration: 3s;  
  
    animation-delay: 2s;
```



```
animation-fill-mode: both;
```

```
}
```

CSS Animation Shorthand Property

The example below uses six of the animation properties:

```
div {
```

```
animation-name: myAnimation;
```

```
animation-duration: 5s;
```

```
animation-timing-function: linear;
```

```
animation-delay: 2s;
```

```
animation-iteration-count: infinite;
```

```
animation-direction: alternate;
```

```
}
```

What is transform?

The transform property is used to change the appearance or position of an element.

It can:

- Move an element
- Rotate an element
- Make it bigger (scale)
- Make it smaller
- Tilt (skew) it

Syntax

transform: value;

Example:

transform: translateX(100px);

This means:

Move the element 100 pixels towards the right.

Properties -

translateX(100px) → Moves the element **100px to the right**.

translateY(100px) → Moves the element **100px downward**.

translate(100px, 50px) → Moves the element **100px to the right** and **50px downward**.

rotate(45deg) → Rotates the element **45° clockwise**.

scale(2) → Makes the element **2 times (double)** its original size.

scale(0.5) → Reduces the element to **half (50%)** of its original size.

skewX(20deg) → Tilts (slants) the element **20° horizontally**.

skewY(20deg) → Tilts (slants) the element **20° vertically**.